Fundamentals of Programming

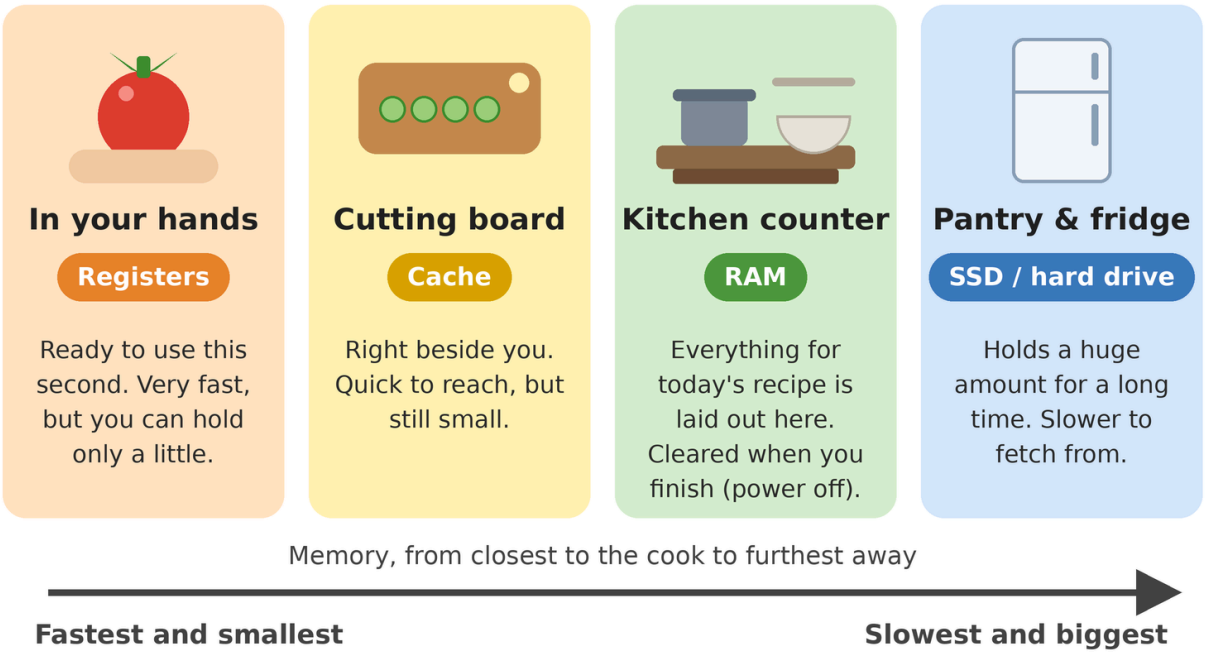
Everything from three years of CSE, explained in simple words

Introduction

A computer is built in layers, called levels. Programming languages work at the upper layers (Levels 4 and 5), the ones humans can read and write. At the bottom, Level 0, is the physical hardware logic of the CPU (the processor, the computer's brain), which only understands electrical signals written as 1s and 0s. Programming languages act as an automated translation pipeline, bridging the gap between human logic and that hardware logic. The purposes of programming languages are as follows:

- **Communicating with the Computer:** They translate human intent into structured instructions the CPU can execute.
- **Task Automation:** They allow developers to write scripts that eliminate tedious, repetitive tasks.
- **Problem Solving:** They let us break complex problems into smaller, manageable steps.
- **Digital Infrastructure:** Software written in them powers everything from mobile apps to operating systems.

A computer also keeps information in different places, a bit like a cook in a kitchen. Registers are the ingredients in your hands and cache is the cutting board: extremely fast to reach, but tiny. RAM is the kitchen counter, the main working space: fast, but it is cleared when the power goes off. Storage such as an SSD or hard drive is the pantry and fridge: slower to fetch from, but much larger, and it keeps your data even when the computer is off. The closer to the cook, the faster the access and the smaller the space.



Computer memory explained as a kitchen

The four core ideas of programming

At its heart, Computer Science is the art of solving problems using a very fast, but completely clueless, assistant (the computer). Here are the four fundamental concepts behind almost everything in programming, in plain English:

1. Data (The Ingredients)

A computer doesn't know what a "user", a "photo", or a "song" is. It only knows information, which we call data. To work with data, we store it in labeled boxes called variables.

- Imagine a labeled box on a desk named **Age**. Inside, you put the number 20.
- Another box named **Username** holds the text "Alex".
- Every time the computer needs to check how old you are, it opens the box labeled Age.

2. Algorithms (The Recipe)

An algorithm is simply a step-by-step set of instructions to complete a task. You already use algorithms every day! Example: making toast.

I. Take a slice of bread.
II. Put it in the toaster.
III. Push the lever down.
IV. Wait 2 minutes.
V. Remove the toast.

If you miss step 2, the algorithm fails. Computers need every single step spelled out in exact order because they cannot "fill in the blanks" like humans do.

3. Control Flow (Making Decisions)

A computer needs to know what to do when situations change. We guide it using if / else statements (logic) and loops (repetition). You will see both in detail later, in the Conditionals and Loop sections.

Logic (if / else):

- If the alarm time is 7:00 AM ⟶ ring the alarm.
- Else (if it is not 7:00 AM) ⟶ stay quiet.

Loops (repetition):
Instead of telling the computer "Check email, check email, check email", you tell it:

- Repeat checking for new messages until a new email arrives.

4. Functions (The Recipe Cards)

Once you write a great set of instructions, say how to send a message, you don't want to rewrite those steps every single time someone clicks "Send". You package those steps into a function and give it a name, like SendMessage(). Anytime you need that action performed, you just call its name, and the computer runs the saved steps automatically.

How they all fit together

Imagine building a basic vending machine:

Concept	How the vending machine uses it
Data (Variables)	MoneyInserted = $2.00, ItemPrice = $1.50
Algorithm	The steps in order: take the money, check it, give the snack, give change.
Logic (if / else)	If MoneyInserted is greater than or equal to ItemPrice, dispense the snack. Else, ask for more money.
Loop	Keep flashing the light until a button is pressed.
Function	GiveChange() calculates and drops the leftover coins.

Programming languages (like Python or JavaScript) are just the specific words and grammar we use to translate these simple ideas into instructions the machine understands.

Print

Let's say you want to read something from a paper, so you print the document using a printer. Now imagine the screen is the paper and your computer is the printer. To read your statement on the screen, you need to tell the computer to print it, otherwise nothing will appear. Different languages use different syntax for this command. For example:

C: `printf("Hello");`

C++: `std::cout << "Hello";`
Java: `System.out.println("Hello");`
Python: `print("Hello")`
JavaScript (JS): `console.log("Hello");`
SQL: `SELECT 'Hello';`
PHP: `echo "Hello";`
Kotlin: `println("Hello")`
Dart: `print('Hello');`
Go: `fmt.Println("Hello")`
TypeScript (TS): `console.log("Hello");`
R: `cat("Hello")`
C#: `Console.WriteLine("Hello");`
MIPS: `la $a0, msg, then li $v0, 4, then syscall`

Note: in MIPS, msg is a label for the stored text "Hello", and 4 is the code that means "print a string". Some languages (like Java and C) also need a little extra setup around these lines, so here we only show the printing part.

Syntax

The term "syntax" comes from the Ancient Greek word σύνταξις (syntaxis), meaning "arrangement together" or "coordination". In linguistics, syntax studies how words and morphemes combine to form larger units like phrases and sentences, focusing on word order, grammatical relations, and hierarchical sentence structure. Different languages have different syntactic rules; for instance, English typically follows a Subject-Verb-Object (SVO) order.

Likewise, in programming, syntax is the set of rules that define how code must be written: its structure, order, and valid combinations of symbols, keywords, and punctuation, so that a computer can correctly interpret and execute it. If the syntax is wrong, a syntax error occurs, just like a grammar mistake in a sentence.

The syntax of programming languages is built on key components:

1. **Keywords:** words reserved by the language that have a fixed meaning (e.g. if, while, return).
2. **Operators:** symbols that work on values (e.g. +, -, ==).
3. **Punctuation Marks:** brackets, braces, semicolons and similar symbols that give code its structure.
4. **Identifiers:** names you choose for things like variables and functions (e.g. age, calculateTotal).
5. **Data Types:** the kind of value being stored, such as whole numbers, decimals, text, or true/false.
6. **Control Structures:** tools that direct the flow of the program, including conditionals, loops, and switch branches.
7. **Comments:** notes written for humans that the computer ignores.

The importance of syntax in programming cannot be overstated. Here's why it matters:

- **Clarity and Readability:** Clean and well-structured syntax enhances code readability, making it easier for developers to understand, maintain, and debug code.
- **Correctness and Accuracy:** Accurate syntax ensures that instructions are interpreted correctly by the computer, reducing the likelihood of errors and bugs in the code.
- **Consistency and Maintainability:** Consistent syntax conventions across a codebase facilitate collaboration among developers and simplify code maintenance and updates.
- **Early Error Detection:** The computer checks syntax before running your code, so mistakes are caught immediately instead of causing strange behavior later.

Conditionals

Assume you want to go on a hangout but your mom says, "You can go only if your friend goes, else you can't." This is a conditional statement. You can express it in two ways: if-else and switch-case.

if-else: if holds the condition and what happens when it is true, and else holds what happens when it is not.

switch-case: switch holds the variable being checked, each case is one possible value with its own consequence, and default covers everything else (it plays the role of else). Switch is handy when there are many fixed options, like days of the week.

Below are the same two examples written in every language from the Print section. In every if-else example, "friend goes" is true, so the program says "Go hangout". In every switch example, the day is 2, so it says "Tuesday".

C

if-else

```
int friendGoes = 1;
if (friendGoes) {
    printf("Go hangout");
} else {
    printf("Stay home");
}
```

switch-case

```
int day = 2;
switch (day) {
    case 1: printf("Monday"); break;
    case 2: printf("Tuesday"); break;
    default: printf("Another day");
}
```

C++

if-else

```
bool friendGoes = true;
if (friendGoes) {
    std::cout << "Go hangout";
} else {
    std::cout << "Stay home";
}
```

switch-case

```
int day = 2;
switch (day) {
    case 1: std::cout << "Monday"; break;
    case 2: std::cout << "Tuesday"; break;
    default: std::cout << "Another day";
}
```

Java

if-else

```
boolean friendGoes = true;
if (friendGoes) {
    System.out.println("Go hangout");
} else {
    System.out.println("Stay home");
}
```

switch-case

```
int day = 2;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    default: System.out.println("Another day");
}
```

Python

if-else

```
friend_goes = True
if friend_goes:
    print("Go hangout")
else:
    print("Stay home")
```

switch-case

```
day = 2
match day:
    case 1:
        print("Monday")
    case 2:
        print("Tuesday")
    case _:
        print("Another day")
```

Python calls its switch "match-case" (Python 3.10 or newer). The underscore _ works as the default.

JavaScript (JS)

if-else

```
let friendGoes = true;
if (friendGoes) {
    console.log("Go hangout");
} else {
    console.log("Stay home");
}
```

switch-case

```
let day = 2;
switch (day) {
    case 1: console.log("Monday"); break;
    case 2: console.log("Tuesday"); break;
    default: console.log("Another day");
}
```

SQL

SQL is mainly for asking questions of a database, so it has no true if-else or loops in its standard form. The closest tools are shown below. The while loop uses T-SQL, the version of SQL used by Microsoft SQL Server.

if-else

```
SELECT CASE
    WHEN 1 = 1 THEN 'Go hangout'
    ELSE 'Stay home'
END;
```

CASE ... WHEN ... ELSE plays the role of if-else. Here 1 = 1 stands for "friend goes" (always true).

switch-case

```
SELECT CASE 2
    WHEN 1 THEN 'Monday'
    WHEN 2 THEN 'Tuesday'
    ELSE 'Another day'
END;
```

PHP

if-else

```
<?php
$friendGoes = true;
if ($friendGoes) {
    echo "Go hangout";
} else {
    echo "Stay home";
}
```

switch-case

```
<?php
$day = 2;
switch ($day) {
    case 1: echo "Monday"; break;
    case 2: echo "Tuesday"; break;
    default: echo "Another day";
}
```

Kotlin

if-else

```
val friendGoes = true
if (friendGoes) {
    println("Go hangout")
} else {
    println("Stay home")
}
```

switch-case

```
val day = 2
when (day) {
    1 -> println("Monday")
    2 -> println("Tuesday")
    else -> println("Another day")
}
```

Kotlin calls its switch "when", and uses else as the default.

Dart

if-else

```
bool friendGoes = true;
if (friendGoes) {
  print('Go hangout');
} else {
  print('Stay home');
}
```

switch-case

```
int day = 2;
switch (day) {
  case 1: print('Monday'); break;
  case 2: print('Tuesday'); break;
  default: print('Another day');
}
```

Go

if-else

```
friendGoes := true
if friendGoes {
    fmt.Println("Go hangout")
} else {
    fmt.Println("Stay home")
}
```

switch-case

```
day := 2
switch day {
case 1:
    fmt.Println("Monday")
case 2:
    fmt.Println("Tuesday")
default:
    fmt.Println("Another day")
}
```

Go stops after the matching case by itself, so no break is needed.

TypeScript (TS)

TypeScript is JavaScript plus type labels (like : boolean), so the structure is the same.

if-else

```
let friendGoes: boolean = true;
if (friendGoes) {
    console.log("Go hangout");
} else {
    console.log("Stay home");
}
```

switch-case

```
let day: number = 2;
switch (day) {
    case 1: console.log("Monday"); break;
    case 2: console.log("Tuesday"); break;
    default: console.log("Another day");
}
```

R

if-else

```
friend_goes <- TRUE
if (friend_goes) {
  cat("Go hangout\n")
} else {
  cat("Stay home\n")
}
```

switch-case

```
day <- "Tue"
switch(day,
  Mon = cat("Monday\n"),
  Tue = cat("Tuesday\n"),
  cat("Another day\n")
)
```

R's switch picks the option whose name matches. The last item, with no name, is the default.

C#

if-else

```
bool friendGoes = true;
if (friendGoes)
{
    Console.WriteLine("Go hangout");
}
else
{
    Console.WriteLine("Stay home");
}
```

switch-case

```
int day = 2;
switch (day)
{
    case 1: Console.WriteLine("Monday"); break;
    case 2: Console.WriteLine("Tuesday"); break;
    default: Console.WriteLine("Another day"); break;
}
```

MIPS

MIPS is assembly language, which is very close to what the processor actually understands. It has no if, else or loop keywords. Instead, you put labels (names for places in the code) and jump to them when a condition is true. Lines starting with # are comments.

if-else

```
.data
msgGo:   .asciiz "Go hangout"
msgStay: .asciiz "Stay home"
.text
main:
    li  $t0, 1              # friendGoes (1 = yes, 0 = no)
    beq $t0, $zero, stay    # if friendGoes is 0, jump to stay
    la  $a0, msgGo          # otherwise prepare "Go hangout"
    j   show
stay:
    la  $a0, msgStay        # prepare "Stay home"
show:
    li  $v0, 4              # 4 = print a string
    syscall
    li  $v0, 10             # 10 = end the program
    syscall
```

switch-case

```
.data
mon:   .asciiz "Monday"
tue:   .asciiz "Tuesday"
other: .asciiz "Another day"
.text
main:
    li  $t0, 2              # day = 2
    li  $t1, 1
    beq $t0, $t1, case1     # is day 1?
    li  $t1, 2
    beq $t0, $t1, case2     # is day 2?
    la  $a0, other          # default
    j   show
case1:
    la  $a0, mon
    j   show
case2:
    la  $a0, tue
show:
    li  $v0, 4
    syscall
    li  $v0, 10
    syscall
```

A switch is just a chain of comparisons, each one jumping to its own label. The last line before the labels is the default.

Loop

Suppose a teacher punishes you by making you write "I am sorry" 100 times. Writing the same thing over and over is annoying. The same goes for playing a turn-based game like chess, counting from 1 to 100, or adding up a list of expenses. These are all repetition problems, and loops exist to solve them easily. The main types are for, while, do-while, and for-each.

- **for loop:** needs a starting point, an ending condition, and a step (how the counter changes each round).
- **while loop:** needs only a condition. It checks the condition first, so if the condition is false from the start, it never runs.
- **do-while loop:** runs the code once first, then checks the condition to decide whether to repeat.
- **for-each loop:** needs a collection (like a list). It runs once for every item in it, with no counting required.

Below are the examples in every language. The for, while and do-while examples each print "I am sorry" three times. The for-each example prints every fruit in a list of three. Where a language is missing a loop type, a note explains the closest alternative.

C

for loop

```
for (int i = 1; i <= 3; i++) {
    printf("I am sorry\n");
}
```

while loop

```
int i = 1;
while (i <= 3) {
    printf("I am sorry\n");
    i++;
}
```

do-while loop

```
int i = 1;
do {
    printf("I am sorry\n");
    i++;
} while (i <= 3);
```

for-each loop

```
char *fruits[] = {"apple", "banana", "mango"};
for (int i = 0; i < 3; i++) {
    printf("%s\n", fruits[i]);
}
```

C has no built-in for-each, so we walk through the list position by position.

C++

for loop

```
for (int i = 1; i <= 3; i++) {
    std::cout << "I am sorry\n";
}
```

while loop

```
int i = 1;
while (i <= 3) {
    std::cout << "I am sorry\n";
    i++;
}
```

do-while loop

```
int i = 1;
do {
    std::cout << "I am sorry\n";
    i++;
} while (i <= 3);
```

for-each loop

```
std::string fruits[] = {"apple", "banana", "mango"};
for (std::string f : fruits) {
    std::cout << f << "\n";
}
```

Java

for loop

```
for (int i = 1; i <= 3; i++) {
    System.out.println("I am sorry");
}
```

while loop

```
int i = 1;
while (i <= 3) {
    System.out.println("I am sorry");
    i++;
}
```

do-while loop

```
int i = 1;
do {
    System.out.println("I am sorry");
    i++;
} while (i <= 3);
```

for-each loop

```
String[] fruits = {"apple", "banana", "mango"};
for (String f : fruits) {
    System.out.println(f);
}
```

Python

for loop

```
for i in range(3):
    print("I am sorry")
```

while loop

```
i = 1
while i <= 3:
    print("I am sorry")
    i += 1
```

do-while loop

```
i = 1
while True:
    print("I am sorry")
    i += 1
    if i > 3:
        break
```

Python has no do-while. The trick is to loop forever and break out at the end of each round once the condition is met.

for-each loop

```
fruits = ["apple", "banana", "mango"]
for f in fruits:
    print(f)
```

In Python, the normal for loop already works as a for-each.

JavaScript (JS)

for loop

```
for (let i = 1; i <= 3; i++) {
    console.log("I am sorry");
}
```

while loop

```
let i = 1;
while (i <= 3) {
    console.log("I am sorry");
    i++;
}
```

do-while loop

```
let i = 1;
do {
    console.log("I am sorry");
    i++;
} while (i <= 3);
```

for-each loop

```
const fruits = ["apple", "banana", "mango"];
for (const f of fruits) {
    console.log(f);
}
```

SQL

SQL is mainly for asking questions of a database, so it has no true if-else or loops in its standard form. The closest tools are shown below. The while loop uses T-SQL, the version of SQL used by Microsoft SQL Server.

for loop

SQL has no for loop. Use the while loop below to count.

while loop

```
DECLARE @i INT = 1;
WHILE @i <= 3
BEGIN
    PRINT 'I am sorry';
    SET @i = @i + 1;
END
```

do-while loop

SQL has no do-while loop.

for-each loop

```
SELECT name FROM fruits;
```

A SELECT already goes through every row of a table, which is the for-each idea. Here, fruits is a table with a name column.

PHP

for loop

```
for ($i = 1; $i <= 3; $i++) {
    echo "I am sorry\n";
}
```

while loop

```
$i = 1;
while ($i <= 3) {
    echo "I am sorry\n";
    $i++;
}
```

do-while loop

```
$i = 1;
do {
    echo "I am sorry\n";
    $i++;
} while ($i <= 3);
```

for-each loop

```
$fruits = ["apple", "banana", "mango"];
foreach ($fruits as $f) {
    echo $f . "\n";
}
```

Kotlin

for loop

```
for (i in 1..3) {
    println("I am sorry")
}
```

while loop

```
var i = 1
while (i <= 3) {
    println("I am sorry")
    i++
}
```

do-while loop

```
var i = 1
do {
    println("I am sorry")
    i++
} while (i <= 3)
```

for-each loop

```
val fruits = listOf("apple", "banana", "mango")
for (f in fruits) {
    println(f)
}
```

Dart

for loop

```
for (int i = 1; i <= 3; i++) {
  print('I am sorry');
}
```

while loop

```
int i = 1;
while (i <= 3) {
  print('I am sorry');
  i++;
}
```

do-while loop

```
int i = 1;
do {
  print('I am sorry');
  i++;
} while (i <= 3);
```

for-each loop

```
var fruits = ['apple', 'banana', 'mango'];
for (var f in fruits) {
  print(f);
}
```

Go

for loop

```
for i := 1; i <= 3; i++ {
    fmt.Println("I am sorry")
}
```

while loop

```
i := 1
for i <= 3 {
    fmt.Println("I am sorry")
    i++
}
```

Go has only one loop keyword, for. Giving it just a condition makes it behave like a while loop.

do-while loop

```
i := 1
for {
    fmt.Println("I am sorry")
    i++
    if i > 3 {
        break
    }
}
```

Go has no do-while. A for loop with no condition runs forever, and break ends it once we are done.

for-each loop

```
fruits := []string{"apple", "banana", "mango"}
for _, f := range fruits {
    fmt.Println(f)
}
```

range goes through the list. The underscore _ ignores the position number, since we only want the item.

TypeScript (TS)

TypeScript is JavaScript plus type labels (like : boolean), so the structure is the same.

for loop

```
for (let i: number = 1; i <= 3; i++) {
    console.log("I am sorry");
}
```

while loop

```
let i: number = 1;
while (i <= 3) {
    console.log("I am sorry");
    i++;
}
```

do-while loop

```
let i: number = 1;
do {
    console.log("I am sorry");
    i++;
} while (i <= 3);
```

for-each loop

```
const fruits: string[] = ["apple", "banana", "mango"];
for (const f of fruits) {
    console.log(f);
}
```

R

for loop

```
for (i in 1:3) {
  cat("I am sorry\n")
}
```

while loop

```
i <- 1
while (i <= 3) {
  cat("I am sorry\n")
  i <- i + 1
}
```

do-while loop

```
i <- 1
repeat {
  cat("I am sorry\n")
  i <- i + 1
  if (i > 3) break
}
```

R has no do-while. repeat runs forever until break stops it, which gives the same effect.

for-each loop

```
fruits <- c("apple", "banana", "mango")
for (f in fruits) {
  cat(f, "\n")
}
```

C#

for loop

```
for (int i = 1; i <= 3; i++)
{
    Console.WriteLine("I am sorry");
}
```

while loop

```
int i = 1;
while (i <= 3)
{
    Console.WriteLine("I am sorry");
    i++;
}
```

do-while loop

```
int i = 1;
do
{
    Console.WriteLine("I am sorry");
    i++;
} while (i <= 3);
```

for-each loop

```
string[] fruits = { "apple", "banana", "mango" };
foreach (string f in fruits)
{
    Console.WriteLine(f);
}
```

MIPS

MIPS is assembly language, which is very close to what the processor actually understands. It has no if, else or loop keywords. Instead, you put labels (names for places in the code) and jump to them when a condition is true. Lines starting with # are comments.

for loop

```
.data
msg: .asciiz "I am sorry\n"
.text
main:
    li  $t0, 1              # i = 1
    li  $t1, 3              # last value
loop:
    bgt $t0, $t1, done      # if i > 3, leave the loop
    li  $v0, 4
    la  $a0, msg
    syscall
    addi $t0, $t0, 1        # i++
    j   loop                # go back to the top
done:
    li  $v0, 10
    syscall
```

while loop

```
.data
msg: .asciiz "I am sorry\n"
.text
main:
    li  $t0, 1
    li  $t1, 3
loop:
    bgt $t0, $t1, done      # check the condition first
    li  $v0, 4
    la  $a0, msg
    syscall
    addi $t0, $t0, 1
    j   loop
done:
    li  $v0, 10
    syscall
```

In assembly, for and while look the same: check, run, jump back.

do-while loop

```
.data
msg: .asciiz "I am sorry\n"
.text
main:
    li  $t0, 1
    li  $t1, 3
loop:
    li  $v0, 4              # run the body first
    la  $a0, msg
    syscall
    addi $t0, $t0, 1
    ble $t0, $t1, loop      # then repeat while i <= 3
    li  $v0, 10
    syscall
```

for-each loop

```
.data
nums: .word 10, 20, 30
.text
main:
    la  $t0, nums           # address of the first item
    li  $t1, 0              # how many items we have visited
    li  $t2, 3              # how many items there are
loop:
    beq $t1, $t2, done
    lw  $a0, 0($t0)         # load the current item
    li  $v0, 1              # 1 = print a number
    syscall
    li  $v0, 11             # 11 = print a character
    li  $a0, 10             # 10 = new line
    syscall
    addi $t0, $t0, 4        # move to the next item
    addi $t1, $t1, 1
    j   loop
done:
    li  $v0, 10
    syscall
```

MIPS has no for-each, so we move a pointer through a list of numbers, one item (4 bytes) at a time.

Array

An array is a set of data of the same type, stored together side by side, like a row of boxes of equal size. It can hold any data type (characters, integers, decimals and so on), but in most languages every item in the array must be of the same type. Each box has a position number, called an index, and counting usually starts at 0, so the first item sits at index 0. (A few languages, like Python and JavaScript, are more relaxed and let one list hold different types.)

To insert data into an array or print all of it, you can use a loop, which visits one box after another. In the examples below, the array holds 10, 20 and 30. We print the first item, then every item.

C

create and print

```
int numbers[3] = {10, 20, 30};
printf("%d\n", numbers[0]);           // first item: 10
for (int i = 0; i < 3; i++) {
    printf("%d\n", numbers[i]);       // 10, 20, 30
}
```

C++

create and print

```
int numbers[3] = {10, 20, 30};
std::cout << numbers[0] << "\n";      // first item: 10
for (int n : numbers) {
    std::cout << n << "\n";           // 10, 20, 30
}
```

Java

create and print

```
int[] numbers = {10, 20, 30};
System.out.println(numbers[0]);       // first item: 10
for (int n : numbers) {
    System.out.println(n);            // 10, 20, 30
}
```

Python

create and print

```
numbers = [10, 20, 30]
print(numbers[0])                     # first item: 10
for n in numbers:
    print(n)                          # 10, 20, 30
```

Python calls its arrays "lists". A list can even mix different types.

JavaScript (JS)

create and print

```
const numbers = [10, 20, 30];
console.log(numbers[0]);              // first item: 10
for (const n of numbers) {
    console.log(n);                   // 10, 20, 30
}
```

A JavaScript array can mix different types.

SQL

create and print

```
CREATE TABLE numbers (value INT);
INSERT INTO numbers VALUES (10), (20), (30);
SELECT value FROM numbers;            -- 10, 20, 30
```

Standard SQL has no arrays. A table with one column is the closest thing, and SELECT walks through every row. (PostgreSQL does have real arrays.)

PHP

create and print

```
$numbers = [10, 20, 30];
echo $numbers[0] . "\n";              // first item: 10
foreach ($numbers as $n) {
    echo $n . "\n";                   // 10, 20, 30
}
```

Kotlin

create and print

```
val numbers = intArrayOf(10, 20, 30)
println(numbers[0])                   // first item: 10
for (n in numbers) {
    println(n)                        // 10, 20, 30
}
```

Dart

create and print

```
List<int> numbers = [10, 20, 30];
print(numbers[0]);                    // first item: 10
for (int n in numbers) {
  print(n);                           // 10, 20, 30
}
```

Dart calls its arrays "lists".

Go

create and print

```
numbers := [3]int{10, 20, 30}
fmt.Println(numbers[0])               // first item: 10
for _, n := range numbers {
    fmt.Println(n)                    // 10, 20, 30
}
```

[3]int means an array of exactly 3 whole numbers.

TypeScript (TS)

create and print

```
const numbers: number[] = [10, 20, 30];
console.log(numbers[0]);              // first item: 10
for (const n of numbers) {
    console.log(n);                   // 10, 20, 30
}
```

R

create and print

```
numbers <- c(10, 20, 30)
cat(numbers[1], "\n")                # first item: 10
for (n in numbers) {
  cat(n, "\n")                       # 10, 20, 30
}
```

R counts from 1, not 0, so numbers[1] is the first item. c() combines values into a list.

C#

create and print

```
int[] numbers = { 10, 20, 30 };
Console.WriteLine(numbers[0]);       // first item: 10
foreach (int n in numbers)
{
    Console.WriteLine(n);            // 10, 20, 30
}
```

MIPS

create and print

```
.data
numbers: .word 10, 20, 30
newline: .asciiz "\n"
.text
main:
    la  $t0, numbers        # address of the array
    lw  $a0, 0($t0)         # first item (position 0)
    li  $v0, 1              # 1 = print a number
    syscall
    la  $a0, newline
    li  $v0, 4
    syscall

    li  $t1, 0              # items visited
    li  $t2, 3              # number of items
loop:
    beq $t1, $t2, done
    lw  $a0, 0($t0)         # load the current item
    li  $v0, 1
    syscall
    la  $a0, newline
    li  $v0, 4
    syscall
    addi $t0, $t0, 4        # next item (each number takes 4 bytes)
    addi $t1, $t1, 1
    j   loop
done:
    li  $v0, 10
    syscall
```

In assembly, an array is just numbers placed side by side in memory. Each number takes 4 bytes, which is why we move 4 at a time.

String

A string is a set of characters stored together in one box. It is basically an array of characters. In some languages (like C) a string really is an array of characters, while in many others it is its own built-in type. Most languages also come with ready-made tools (built-in functions or libraries) for string input, output, reversing, finding the length and more.

In the examples below, the text is "Hello". We print its length (5) and then the text reversed (olleH).

C

length and reverse

```c
#include <string.h>

char s[] = "Hello";
printf("%d\n", (int)strlen(s));       // length: 5
for (int i = (int)strlen(s) - 1; i >= 0; i--) {
    printf("%c", s[i]);               // reversed: olleH
}
```

In C, a string really is an array of characters, ending with a hidden 0 that marks the end.

C++

length and reverse

```cpp
std::string s = "Hello";
std::cout << s.length() << "\n";      // length: 5
std::string r(s.rbegin(), s.rend());  // reversed copy
std::cout << r;                       // olleH
```

Java

length and reverse

```java
String s = "Hello";
System.out.println(s.length());                        // 5
System.out.println(new StringBuilder(s).reverse());    // olleH
```

Python

length and reverse

```python
s = "Hello"
print(len(s))                         # length: 5
print(s[::-1])                        # reversed: olleH
```

[::-1] means "walk through the text backwards".

JavaScript (JS)

length and reverse

```javascript
const s = "Hello";
console.log(s.length);                          // 5
console.log(s.split("").reverse().join(""));   // olleH
```

SQL

length and reverse

```
SELECT LENGTH('Hello'), REVERSE('Hello');   -- 5, olleH
```

This works in MySQL and PostgreSQL. In SQL Server, use LEN instead of LENGTH.

PHP

length and reverse

```
$s = "Hello";
echo strlen($s) . "\n";               // length: 5
echo strrev($s);                      // reversed: olleH
```

Kotlin

length and reverse

```
val s = "Hello"
println(s.length)                     // length: 5
println(s.reversed())                 // reversed: olleH
```

Dart

length and reverse

```
String s = 'Hello';
print(s.length);                      // length: 5
print(s.split('').reversed.join());   // reversed: olleH
```

Go

length and reverse

```
s := "Hello"
fmt.Println(len(s))                   // length: 5
r := []rune(s)
for i, j := 0, len(r)-1; i < j; i, j = i+1, j-1 {
    r[i], r[j] = r[j], r[i]           // swap the ends, moving inward
}
fmt.Println(string(r))                // reversed: olleH
```

Go has no ready-made reverse, so we swap letters from both ends.

TypeScript (TS)

length and reverse

```
const s: string = "Hello";
console.log(s.length);                          // 5
console.log(s.split("").reverse().join(""));   // olleH
```

R

length and reverse

```
s <- "Hello"
cat(nchar(s), "\n")                   # length: 5
cat(paste(rev(strsplit(s, "")[[1]]), collapse = ""), "\n")   # olleH
```

The reverse line splits the text into letters, flips their order, and joins them back.

C#

length and reverse

```
string s = "Hello";
Console.WriteLine(s.Length);          // length: 5
char[] letters = s.ToCharArray();
Array.Reverse(letters);
Console.WriteLine(new string(letters));   // reversed: olleH
```

MIPS

length and reverse

```
.data
str: .asciiz "Hello"
.text
main:
    la  $t0, str            # address of the first character
    li  $t1, 0              # length counter
loop:
    lb  $t2, 0($t0)         # load one character
    beq $t2, $zero, done    # a 0 marks the end of the text
    addi $t1, $t1, 1
    addi $t0, $t0, 1
    j   loop
done:
    move $a0, $t1
    li  $v0, 1              # print the length: 5
    syscall
    li  $v0, 10
    syscall
```

MIPS has no ready-made string tools, so we count the characters ourselves. Reversing would need even more code, so it is left out here.

Function

In programming, a function is simply a reusable block of code designed to perform a specific task. Think of a function like a recipe or a shortcut button on your microwave. Instead of writing out a long list of instructions every single time you want to do something, you group those instructions together, give them a name, and press that "button" whenever you need them.

A function can also take inputs (called parameters) and give back an answer (the return value), like putting bread in a toaster and getting toast out. In the examples below, the function adds two numbers: we give it 2 and 3, and it gives back 5.

C

define and call

```c
int add(int a, int b) {
    return a + b;
}

int main() {
    printf("%d", add(2, 3));          // 5
    return 0;
}
```

C++

define and call

```cpp
int add(int a, int b) {
    return a + b;
}

int main() {
    std::cout << add(2, 3);           // 5
    return 0;
}
```

Java

define and call

```java
public static int add(int a, int b) {
    return a + b;
}

// then, inside main:
System.out.println(add(2, 3));        // 5
```

In Java, functions live inside a class and are called methods.

Python

define and call

```python
def add(a, b):
    return a + b

print(add(2, 3))                      # 5
```

JavaScript (JS)

define and call

```javascript
function add(a, b) {
    return a + b;
}

console.log(add(2, 3));               // 5
```

SQL

define and call

```
CREATE FUNCTION add_numbers (@a INT, @b INT)
RETURNS INT
AS
BEGIN
    RETURN @a + @b;
END;
GO

SELECT dbo.add_numbers(2, 3);          -- 5
```

This is T-SQL (SQL Server). The way functions are written differs a lot between databases.

PHP

define and call

```
function add($a, $b) {
    return $a + $b;
}

echo add(2, 3);                        // 5
```

Kotlin

define and call

```
fun add(a: Int, b: Int): Int {
    return a + b
}

println(add(2, 3))                     // 5
```

Dart

define and call

```
int add(int a, int b) {
  return a + b;
}

print(add(2, 3));                      // 5
```

Go

define and call

```
func add(a int, b int) int {
    return a + b
}

fmt.Println(add(2, 3))                 // 5
```

TypeScript (TS)

define and call

```
function add(a: number, b: number): number {
    return a + b;
}

console.log(add(2, 3));               // 5
```

R

define and call

```
add <- function(a, b) {
  return(a + b)
}

print(add(2, 3))                      # 5
```

C#

define and call

```
static int Add(int a, int b)
{
    return a + b;
}

Console.WriteLine(Add(2, 3));         // 5
```

MIPS

define and call

```
.text
main:
    li  $a0, 2              # first input
    li  $a1, 3              # second input
    jal add                 # call the function
    move $a0, $v0           # take the answer
    li  $v0, 1              # print it: 5
    syscall
    li  $v0, 10
    syscall

add:
    add $v0, $a0, $a1       # the answer goes in $v0
    jr  $ra                 # go back to where we were called from
```

jal means "jump and remember where I came from", and jr $ra means "go back".

Pointers

A pointer is a variable that stores the memory address of another variable instead of holding the data value directly.

The House Address Analogy

- **Regular variable:** think of it as a house containing a specific item (like the number 42).
- **Pointer:** think of it as a slip of paper with the house's street address written on it.

Instead of carrying around a heavy box of items (large data), you just carry the tiny slip of paper with the address telling you where the items are kept.

Only some languages (like C, C++, Go and MIPS) let you work with addresses directly. Many others hide them and give you references instead, which work like a second slip of paper with the same address: two variables leading to the same house. In every example below, we start with 42 in the house, change it to 50 through the address (or reference), and check the result.

C

pointer or reference

```c
int x = 42;
int *p = &x;                        // p holds the address of x
printf("%d\n", *p);                 // follow the address: 42
*p = 50;                            // change the value at that address
printf("%d\n", x);                  // 50
```

*& means "the address of", and * means "the value at that address".*

C++

pointer or reference

```cpp
int x = 42;
int *p = &x;                        // p holds the address of x
std::cout << *p << "\n";            // follow the address: 42
*p = 50;                            // change the value at that address
std::cout << x;                     // 50
```

Java

pointer or reference

```java
int[] a = {42};
int[] b = a;                        // b points to the same array as a
b[0] = 50;                          // change it through b
System.out.println(a[0]);           // 50
```

Java has no pointers you can see. Variables holding objects store a reference, which works like a copy of the slip of paper: both slips lead to the same house.

Python

pointer or reference

```python
a = [42]
b = a                               # b points to the same list as a
b[0] = 50                           # change it through b
print(a[0])                         # 50
```

Python has no visible pointers. Variables are references, like slips of paper that lead to the same house.

JavaScript (JS)

pointer or reference

```
const a = [42];
const b = a;                          // b points to the same array as a
b[0] = 50;                            // change it through b
console.log(a[0]);                    // 50
```

JavaScript has no visible pointers. Arrays and objects are passed around as references.

SQL

SQL has no pointers.

PHP

pointer or reference

```
$x = 42;
$p = &$x;                             // $p is another name for the same value as $x
$p = 50;                              // change it through $p
echo $x;                              // 50
```

PHP has no real pointers, but & creates a reference: a second name for the same house.

Kotlin

pointer or reference

```
val a = intArrayOf(42)
val b = a                             // b points to the same array as a
b[0] = 50                             // change it through b
println(a[0])                         // 50
```

Kotlin has no visible pointers. Variables holding objects are references.

Dart

pointer or reference

```
List<int> a = [42];
var b = a;                            // b points to the same list as a
b[0] = 50;                            // change it through b
print(a[0]);                          // 50
```

Dart has no visible pointers. Variables holding objects are references.

Go

pointer or reference

```
x := 42
p := &x                               // p holds the address of x
fmt.Println(*p)                       // follow the address: 42
*p = 50                               // change the value at that address
fmt.Println(x)                        // 50
```

TypeScript (TS)

pointer or reference

```
const a: number[] = [42];
const b = a;                          // b points to the same array as a
b[0] = 50;                            // change it through b
console.log(a[0]);                    // 50
```

Like JavaScript, TypeScript has no visible pointers. Arrays and objects are passed around as references.

R

pointer or reference

```
e1 <- new.env()
e1$value <- 42
e2 <- e1                              # e2 points to the same environment as e1
e2$value <- 50                        # change it through e2
print(e1$value)                       # 50
```

R has no pointers. Normal values are copied, but an environment is shared, so it behaves like a reference.

C#

pointer or reference

```
int[] a = { 42 };
int[] b = a;                          // b points to the same array as a
b[0] = 50;                            // change it through b
Console.WriteLine(a[0]);              // 50
```

C# normally hides pointers and uses references. Real pointers exist only inside special "unsafe" code.

MIPS

pointer or reference

```
.data
x: .word 42
.text
main:
    la  $t0, x              # $t0 holds the address of x (the pointer)
    lw  $a0, 0($t0)         # follow the address and read: 42
    li  $v0, 1
    syscall
    li  $v0, 11             # print a new line
    li  $a0, 10
    syscall
    li  $t1, 50
    sw  $t1, 0($t0)         # follow the address and write 50
    lw  $a0, 0($t0)         # read again: 50
    li  $v0, 1
    syscall
    li  $v0, 10
    syscall
```

In assembly you work with addresses all the time, so every la/lw/sw line is pointer work.

Structures

Think of a structure like the blueprint for a building. The blueprint lays out which rooms (pieces of data) the building has, and every building made from it follows the same layout. In the same way, a structure is a blueprint for a piece of your software: it groups several related pieces of data, which can be of different types, under one name. This lets you create your own custom data types.

A structure is not a function. A function does something, while a structure holds data. In the examples below, a Student blueprint holds a name (text) and an age (number). We make one student, Ana, aged 20, and print the details.

C

define and use

```
struct Student {
    char name[20];
    int age;
};

struct Student s = {"Ana", 20};
printf("%s is %d", s.name, s.age);    // Ana is 20
```

C++

define and use

```
struct Student {
    std::string name;
    int age;
};

Student s = {"Ana", 20};
std::cout << s.name << " is " << s.age;   // Ana is 20
```

Java

define and use

```
record Student(String name, int age) {}

Student s = new Student("Ana", 20);
System.out.println(s.name() + " is " + s.age());   // Ana is 20
```

Java has no struct. A record (Java 16 or newer) or a class does the same job.

Python

define and use

```
from dataclasses import dataclass

@dataclass
class Student:
    name: str
    age: int

s = Student("Ana", 20)
print(s.name, "is", s.age)            # Ana is 20
```

Python has no struct. A dataclass is the simplest way to make a custom data type.

JavaScript (JS)

define and use

```
const s = { name: "Ana", age: 20 };
console.log(s.name + " is " + s.age); // Ana is 20
```

JavaScript has no struct. An object groups named pieces of data in the same way.

SQL

define and use

```
CREATE TABLE Student (
    name VARCHAR(20),
    age INT
);
INSERT INTO Student VALUES ('Ana', 20);
SELECT name, age FROM Student;        -- Ana, 20
```

In SQL, a table is the blueprint, and each row is one building made from it.

PHP

define and use

```
class Student {
    public $name;
    public $age;
}

$s = new Student();
$s->name = "Ana";
$s->age = 20;
echo $s->name . " is " . $s->age;     // Ana is 20
```

PHP has no struct. A class does the same job.

Kotlin

define and use

```
data class Student(val name: String, val age: Int)

val s = Student("Ana", 20)
println(s.name + " is " + s.age)      // Ana is 20
```

Kotlin has no struct. A data class is the simplest way to make a custom data type.

Dart

define and use

```
class Student {
  String name;
  int age;
  Student(this.name, this.age);
}

var s = Student('Ana', 20);
print(s.name + ' is ' + s.age.toString());   // Ana is 20
```

Dart has no struct. A class does the same job.

Go

define and use

```
type Student struct {
    Name string
    Age  int
}

s := Student{"Ana", 20}
fmt.Println(s.Name, "is", s.Age)      // Ana is 20
```

TypeScript (TS)

define and use

```
interface Student {
    name: string;
    age: number;
}

const s: Student = { name: "Ana", age: 20 };
console.log(s.name + " is " + s.age); // Ana is 20
```

An interface is the blueprint, and the object made from it is the building.

R

define and use

```
s <- list(name = "Ana", age = 20)
cat(s$name, "is", s$age, "\n")        # Ana is 20
```

R has no struct. A list with named items does the same job.

C#

define and use

```
struct Student
{
    public string Name;
    public int Age;
}

Student s = new Student { Name = "Ana", Age = 20 };
Console.WriteLine(s.Name + " is " + s.Age);   // Ana is 20
```

MIPS

define and use

```
.data
student:  .word 20          # field 1: age (starts at position 0)
          .asciiz "Ana"     # field 2: name (starts 4 bytes in)
is_text:  .asciiz " is "
.text
main:
    la  $t0, student        # address of the whole structure
    addi $a0, $t0, 4        # the name begins 4 bytes in
    li  $v0, 4
    syscall
    la  $a0, is_text
    li  $v0, 4
    syscall
    lw  $a0, 0($t0)         # the age is at position 0
    li  $v0, 1
    syscall                 # prints: Ana is 20
    li  $v0, 10
    syscall
```

MIPS has no struct. You decide the layout yourself and remember how far into the memory each field sits.

File

A file is a named container for data that is kept on your computer's storage (like the pantry and fridge in our kitchen picture). Variables and arrays live in memory and disappear when the program ends. A file keeps its data even after the program stops or the computer is switched off.

Using a file is like using a paper document. You follow three steps:

1. **Open** the file, saying what you want to do.
2. **Read or write:** take data out of it, or put data into it.
3. **Close** the file when you are done.

When you open a file, you choose a mode:

- **Read:** you can only look at what is inside. If the file does not exist, you get an error.
- **Write:** you start with a blank page. If the file already had text, the old text is erased.
- **Append:** you add new text to the end and keep what was already there.

Many newer languages hide these steps and open and close the file for you. In the examples below, we write one line, "Hello from the file", into a file called notes.txt, then read it back and print it. The file is created in the same folder where the program runs.

C

write and read

```c
FILE *f = fopen("notes.txt", "w");        // open for writing
fprintf(f, "Hello from the file\n");     // write a line
fclose(f);                                // close it

char line[100];
f = fopen("notes.txt", "r");              // open for reading
fgets(line, 100, f);                      // read one line
printf("%s", line);                       // Hello from the file
fclose(f);
```

In C, fopen opens a file, and the letter says what you want to do: w for write, r for read, a for append.

C++

write and read

```cpp
#include <fstream>

std::ofstream out("notes.txt");           // open for writing
out << "Hello from the file\n";           // write a line
out.close();                              // close it

std::ifstream in("notes.txt");            // open for reading
std::string line;
std::getline(in, line);                   // read one line
std::cout << line;                        // Hello from the file
in.close();
```

Java

write and read

```java
import java.nio.file.*;

Files.writeString(Path.of("notes.txt"), "Hello from the file\n");
String text = Files.readString(Path.of("notes.txt"));
System.out.print(text);                   // Hello from the file
```

These shortcuts open and close the file for you (Java 11 or newer). They must sit inside a method that handles IOException, Java's way of reporting file problems.

Python

write and read

```python
with open("notes.txt", "w") as f:         # open for writing
    f.write("Hello from the file\n")

with open("notes.txt", "r") as f:         # open for reading
    print(f.read())                       # Hello from the file
```

A with block closes the file automatically when it ends, so you cannot forget.

JavaScript (JS)

write and read

```
const fs = require("fs");

fs.writeFileSync("notes.txt", "Hello from the file\n");
const text = fs.readFileSync("notes.txt", "utf8");
console.log(text);                          // Hello from the file
```

This is Node.js, which runs JavaScript outside a web page. JavaScript inside a browser cannot freely read or write files on your computer, for safety.

SQL

SQL does not read or write ordinary files itself. A database keeps its data in tables, and the database software manages the files behind the scenes. Importing or exporting files uses commands that differ from one database to another.

PHP

write and read

```
file_put_contents("notes.txt", "Hello from the file\n");
echo file_get_contents("notes.txt");      // Hello from the file
```

These two functions open, use and close the file for you.

Kotlin

write and read

```
import java.io.File

val file = File("notes.txt")
file.writeText("Hello from the file\n")
println(file.readText())                  // Hello from the file
```

Dart

write and read

```
import 'dart:io';

final file = File('notes.txt');
file.writeAsStringSync('Hello from the file\n');
print(file.readAsStringSync());           // Hello from the file
```

File access in Dart works in command-line programs, but not in a web browser.

Go

write and read

```
os.WriteFile("notes.txt", []byte("Hello from the file\n"), 0644)
data, _ := os.ReadFile("notes.txt")
fmt.Print(string(data))                   // Hello from the file
```

0644 is a permission code (the owner can read and write, everyone else can only read). The _ ignores the error value to keep the example short. Needs Go 1.16 or newer and the os package.

TypeScript (TS)

write and read

```
import * as fs from "fs";

fs.writeFileSync("notes.txt", "Hello from the file\n");
const text: string = fs.readFileSync("notes.txt", "utf8");
console.log(text);                          // Hello from the file
```

Like JavaScript, this runs in Node.js, not in a browser.

R

write and read

```
writeLines("Hello from the file", "notes.txt")
text <- readLines("notes.txt")
cat(text, "\n")                             # Hello from the file
```

C#

write and read

```
using System.IO;

File.WriteAllText("notes.txt", "Hello from the file\n");
string text = File.ReadAllText("notes.txt");
Console.Write(text);                        // Hello from the file
```

These shortcuts open and close the file for you.

MIPS

write and read

```
.data
fname:   .asciiz "notes.txt"
message: .asciiz "Hello from the file\n"
buffer:  .space 64               # empty space to hold what we read
.text
main:
    # ---- write ----
    li  $v0, 13                  # 13 = open a file
    la  $a0, fname
    li  $a1, 1                   # 1 = write mode
    li  $a2, 0
    syscall
    move $s0, $v0                # keep the file handle
    li  $v0, 15                  # 15 = write to a file
    move $a0, $s0
    la  $a1, message
    li  $a2, 20                  # 20 characters
    syscall
    li  $v0, 16                  # 16 = close the file
    move $a0, $s0
    syscall

    # ---- read ----
    li  $v0, 13
    la  $a0, fname
    li  $a1, 0                   # 0 = read mode
    li  $a2, 0
    syscall
    move $s0, $v0
    li  $v0, 14                  # 14 = read from a file
    move $a0, $s0
    la  $a1, buffer
    li  $a2, 20
    syscall
    li  $v0, 16
    move $a0, $s0
    syscall

    la  $a0, buffer              # print what we read
    li  $v0, 4
    syscall
    li  $v0, 10
    syscall
```

File syscalls work in simulators like MARS. The file handle returned by "open" is like a ticket number that you hand back for every later step.